



SECCIÓN REGIÓN HUETAR NORTE Y CARIBE
CAMPUS SARAPIQUÍ
TEL: 27646050
CORREO ELECTRÓNICO: sarapiqui@una.ac.cr



Universidad Nacional de Costa Rica
Sección Regional Huetar Norte y Caribe
1º Semestre, 2026

Examen #1: Arquitectura

Carrera:

Ingeniería en Sistemas de la Información

Curso:

Diseño y Programación de Plataformas Móviles

Profesora:

Mag. Rachel Bolívar Morales

Fecha:

25/04/2026

Estudiante:

Adam Acuña González

Tipo de Arquitectura

La arquitectura que se implementará para este proyecto será nativa, de esta manera se asegurara tener una plataforma 100% optimizada para el tipo de sistema operativo en el que se está utilizando, haciendo uso de un lenguaje de programación creado específicamente para el desarrollo de apps en un determinado sistema de operativo, como lo es kotlin para el desarrollo de aplicaciones android. En este contexto, se asegura tener una app que no deje por fuera incluso a dispositivos móviles más antiguos, colocando un SDK mínimo para versiones de Android más antiguas.

El usar este tipo de arquitectura podría dar a pensar que se están dejando por fuera a otras plataformas como IOS o HarmonyOS, sin embargo, el mercado está mayormente dominado por Android, de igual manera el desarrollo de una versión web de la plataforma haría que la misma se pueda utilizar en otros sistemas operativos, incluidas computadoras, y en un futuro se podría, haciendo uso de igual manera de una arquitectura nativa, desarrollar la misma para otros sistemas operativos como los anteriormente mencionados.

Patrón de arquitectura seleccionado

El patrón de arquitectura que se desarrollará para este proyecto será el Model-View-ViewModel (MVVM), pues es el más al día con respecto a las convenciones de la industria, y el que ofrece la arquitectura por capas más óptima cuando se trata de desarrollo móvil en Android, siendo que se divide en las 3 capas que justamente conforman su nombre, la capa Model, en esta se definen la lógica de negocio, y cómo se van a manejar los datos, además de definir de donde se obtienen y almacenan los mismos, luego tenemos la capa View, esta es la interfaz gráfica, en esta capa solo se desarrolla el apartado visual, lo que el usuario ve, se define el diseño y la experiencia de usuario, por último está la capa ViewModel, esta capa, como su nombre lo resalta a la interpretación, es una capa que funciona como puente entre la de View y la de Model, en ella se define como se quieren mostrar los datos al usuario, y que cosas realizar con los datos que el usuario ingresa, manejando la lógica de presentación de los mismos. Con estas capas bien definidas, se mantendrá una estructura ordenada, legible y escalable para el desarrollo del proyecto.

Justificación técnica basada en el problema planteado

El uso de herramientas nativas, como ya se mencionó desde un inicio, permite el desarrollo de una aplicación más optimizada para el dispositivo en el que se va a utilizar, en este caso y de manera inicial dispositivos Android, también, a como ya se mencionó en al principio, se desarrollará la plataforma para dispositivos Android primero debido al dominio del mercado que este Sistema Operativo presenta actualmente, siendo el más utilizado a nivel mundial como se ve en la siguiente cita textual: “La plataforma de Android, por ser el sistema operativo más usado en la actualidad” (Corilla Quispe, 2022, p. 1)

Gracias al centrarse en Android de forma inicial se logrará llegar a más personas en el mercado, de esta manera se podrán identificar nuevas necesidades de los usuarios, logrando desarrollar funcionalidades que las suplan, así como la identificación de problemas tanto técnicos como conceptuales en la misma, puliendo aún más la plataforma en lo que se desarrolla una nueva versión de la misma que llegue a más plataformas tanto móviles como web.

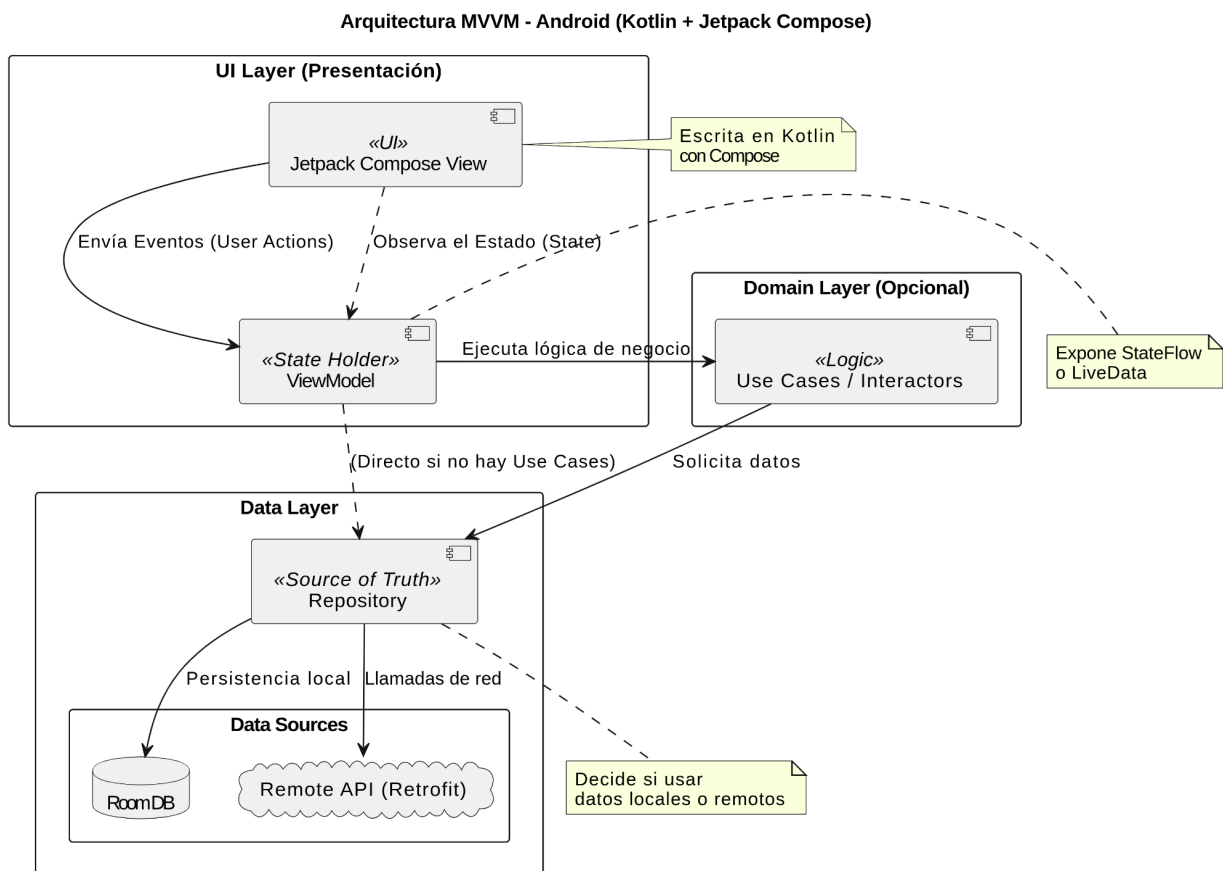
La arquitectura implementada, como se mencionó anteriormente, ofrece un mayor orden, dividiendo el código en 3 capas principales, las cuales ofrecen una maniobrabilidad excelente sobre todo para poder escalar el proyecto y darle mantenimiento.

Ahora bien, ¿por qué usar kotlin como lenguaje de programación en lugar de algún otro lenguaje como lo es el mismo java?, esto por que

Kotlin, siendo un lenguaje moderno, multiplataforma que puede ser utilizado para varios fines, es por esta razón, que los programadores usan para el desarrollo móvil en Android como en iOS; también es conocido por su diseño práctico y su concisa sintaxis, sin olvidar que provee una gran oportunidad para la reutilización de código y compartir entre desarrolladores el uso de múltiples proyectos. (Corilla Quispe, 2022, 3)

Diagrama de arquitectura del sistema

Para ilustrar de manera visual cómo quedará la arquitectura del sistema a nivel móvil, se desarrolló el siguiente diagrama:



En este diagrama se definen también partes de la arquitectura que son fundamentales para el desarrollo de la plataforma, como lo son la tecnología Retrofit con la que se comunicara la plataforma con el API remoto que se deberá de desarrollar, y las bases de datos que se usarán a nivel local dentro de la app de modo que no se sobre cargue el API con llamadas excesivas para el almacenamiento o lectura de datos que bien pueden estar resguardados a nivel local en el dispositivo móvil.

Además, se hace mención también de la tecnología Jetpack Compose, la cuál debe de ser implementada para el desarrollo del apartado gráfico, pues es la que ofrece las mejores prestaciones para la presentación de datos al usuario.

Explicación del flujo general del sistema

Una vez representada la arquitectura de manera visual con el anterior diagrama, resulta más fácil de explicar y de entender el flujo del sistema, por lo que se detalla a continuación.

En primer lugar, el usuario ejecutará una acción en la pantalla del celular, la cual está hecha con Jetpack Compose dentro de la capa de View, esta capa lo que hace es avisarle al ViewModel que el usuario tocó un botón o escribió algo. El ViewModel recibe ese aviso y se encarga de pedir los datos que se ocupan a la capa del Model, específicamente al repositorio, que es el que decide de dónde sacar la información.

Después, el repositorio se fija si ocupa usar Retrofit para traer los datos desde el internet por medio de una API, o si más bien los saca de la base de datos local que está en el mismo teléfono para que sea más rápido. Cuando ya se tiene la información, esta se devuelve al ViewModel, que es donde se acomodan los datos para que se vean bien y se decide qué es lo que se le va a mostrar al usuario en ese momento.

Por último, como la View está siempre escuchando lo que pasa en el ViewModel, la pantalla se actualiza sola y le muestra al usuario la información que pidió. De esta forma, el proceso es circular y muy ordenado, porque cada parte hace su trabajo por separado, logrando que la app no se pegue y que sea más sencillo arreglar cualquier error que salga en el futuro porque ya sabemos exactamente en qué capa buscar.



**SECCIÓN REGIÓN HUETAR NORTE Y CARIBE
CAMPUS SARAPIQUÍ**

TEL: 27646050

CORREO ELECTRÓNICO: sarapiqui@una.ac.cr



References

Corilla Quispe, K. V. (2022, enero-junio). Desarrollo de aplicaciones móviles usando el lenguaje Kotlin. *Dialogos Abierto*, 1(1), 22-23.
<https://doi.org/10.32654/DialogosAbiertos.1-1.3>